

第 10 讲

Introduction to ML; Inference and Training in DNNs

目录

1	机器学习与表示学习	1
1.1	前馈神经网络与课程事项	1
1.2	从输入输出规则到从样本中学习	1
1.3	穷举规则的规模与特征的作用	2
1.4	规则的描述难度与计算难度	3
1.5	人工特征与学习得到的表示	4
2	从感知机到多层网络	6
2.1	分类函数与单个感知机	6
2.2	异或问题与隐藏表示	6
2.3	全连接层、网络层次与参数维度	8
2.4	手写数字识别网络的具体结构	9
3	前向计算与分类输出	11
3.1	输入、分数、概率与类别的区别	11
3.2	激活函数的形状与可求导性	12
3.3	Softmax 与互斥类别的概率	13
4	损失函数、反向传播与参数更新	15
4.1	数字识别例子中的损失函数	15
4.2	从损失求到 softmax 输入的梯度	16
4.3	梯度下降与链式法则	17
4.4	单层平方误差的完整更新	17
4.5	全连接层的三个梯度	18
4.6	两层数字识别网络的完整反向链	19
4.7	前向缓存、梯度传递与参数更新的时序	20
4.8	两层网络的数值核算	21
5	训练组织与并行计算	22
5.1	训练轮次与一次参数更新	22
5.2	全批量与小批量梯度下降	22
5.3	把小批量写成矩阵运算	24
6	泛化、模型容量与任务适配	25
6.1	训练集、验证集与测试集	25
6.2	模型容量、欠拟合与过拟合	26
6.3	从权重图像理解学习到的特征	27
6.4	无免费午餐定理的任务层面含义	27
7	关键公式与结构关系	28
7.1	从输入到参数更新	28
7.2	全连接层的三个梯度	28

1 机器学习与表示学习

1.1 前馈神经网络与课程事项

前馈神经网络 (feedforward neural network) 通过一系列有方向的层, 将输入变换为输出。神经元模型与激活函数决定各层的计算方式, 梯度下降则通过调整参数完成训练。与并行程序设计的联系在于: 网络的前向计算需要大量点积与矩阵运算, 训练还需要计算、传播并汇总这些运算对应的梯度。

课程事项: 2026 年 9 月 24 日

Lab 3 于当周周五截止; 项目里程碑 1 (Project milestone 1) 即将发布。期中考试 1 的时间为 10 月 6 日 19:00–22:00。课后事项为提交 Lab 3、选择项目; 阅读范围为教材第 4 版第 16 章, 或第 5 版第 19 章。

1.2 从输入输出规则到从样本中学习

机器学习 (machine learning) 从数据中提取模式, 用这些模式建立输入与输出之间的关系。对于难以完整写出程序规则的任务, 可以提供带标签的数据, 即已经配对的输入和期望输出, 再迭代调整模型参数, 使模型的答案接近期望答案。这个调整过程称为训练 (training); 参数确定以后, 由新输入计算输出的过程称为推断 (inference)。分类、回归、语音转写和翻译都是机器学习任务。

数据驱动的方法依赖三个条件: 有足够的计算能力反复训练模型, 有能够表达目标关系的数据, 以及存在难以手工完整描述规则的应用需求。GPU 与 CUDA 缩短了深度神经网络的训练与实验周期; 传感器、网络服务和存储设施增加了可获得的数据; 自动驾驶、智能设备、安全和医疗等需求则推动了相关应用。

计算速度提高以后, 网络结构、特征选择、输入输出形式和高质量标注仍然需要设计。训练数据的作用是提供可学习的关系; 数据是否覆盖了任务中的关键变化, 直接影响模型有机会学到什么。

1.3 穷举规则的规模与特征的作用

1.3.1 小规模布尔函数可以由真值表完整指定

对三个二进制输入 a, b, c ，总共只有 $2^3 = 8$ 种输入。表 1 给出一个完整的输入输出关系。只要列出全部输出为 1 的输入组合，就能构造对应的逻辑表达式和逻辑电路。

表 1 三个输入的奇偶性函数。

a	b	c	输出	输入中 1 的个数
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	2
1	0	0	1	1
1	0	1	0	2
1	1	0	0	2
1	1	1	1	3

这张表还具有一个更紧凑的描述：输入中 1 的个数为奇数时输出 1，为偶数时输出 0。相应的布尔表达式是 $a \oplus b \oplus c$ ，其中 $\oplus$ 表示异或。奇偶性就是能够解释全部观测的一个特征。只有部分观测时，例如 $000 \mapsto 0$ 、 $011 \mapsto 0$ 、 $100 \mapsto 1$ 、 $110 \mapsto 0$ ，还不能仅凭查表覆盖所有输入；能够提取这种规律，才有机会对未出现的组合给出答案。

1.3.2 图像输入使查表规模呈指数增长

一张 32×32 的图像包含 1024 个像素。按每像素 8 位计算，输入总共需要 8192 位，因此所有可能输入的数量是 2^{8192} 。即使每种输入只对应一个类别，完整真值表仍然需要为这 2^{8192} 种输入分别记录输出。

因为 $10^9 \approx 2^{30}$ ，十亿个不同输入覆盖的比例约为

$$\frac{2^{30}}{2^{8192}} = 2^{-8162}. \quad (1)$$

这个数量级说明，收集更多样本仍然无法接近穷举整个输入空间。学习过程需要利用多个样本共有的特征，把有限观测中的结构推广到未观测输入。小规模逻辑例子的奇偶性，正好说明了“从大量具体情况中提取简洁规律”这一作用。

1.4 规则的描述难度与计算难度

1.4.1 两个不同的困难来源

一个任务可以容易形式化，却需要很大的计算量；也可以由人较容易完成，却难以写出完整规则。国际象棋的棋盘、棋子与合法走法都很明确，其困难主要来自搜索规模。人脸检测与语音识别则说明了另一种困难：识别结果容易理解，但把所有输入条件转化为固定规则很困难。文学分析与国际外交被用作更复杂任务的示意例子。

这种区分与并行计算有关。对于运算规则已经明确的任务，可以继续研究算法复杂度、并行性与数据带宽；对于输入输出关系本身难以手工表达的任务，还需要解决模型如何从观测中获得这种关系的问题。

1.4.2 国际象棋：规则明确，搜索树很大

棋盘有 64 个位置，初始有 32 个棋子。根据合法走法生成后继局面、为搜索树叶节点评分、再根据这些评分选择路径，就形成了一种可明确描述的计算过程。如果每个局面平均约有 30 种合法走法，搜索 10 个半回合的叶节点数量约为

$$30^{10} = 5.9049 \times 10^{14}. \quad (2)$$

即使每秒处理 10^6 个局面，也需要约 5.9×10^8 秒，即约 18.7 年。因此，启发式方法、剪枝、并行搜索和更快的计算机共同影响这种方案是否实用。Deep Blue 在 1997 年击败 Garry Kasparov，是一个历史例子。

1.4.3 规则系统：常识需要一致的语义表示

规则系统用事实与规则驱动推理引擎 (inference engine)，产生可由已有知识推出的结论。Cyc 的例子把常识组织为本体和知识库，再对形式化陈述进行推理。例如，从“某人是一位美国总统”和“美国总统是美国公民”可以推出该人是美国公民。这说明规则能够显式表达一条推理链。

“Fred 正在刮胡子”的例子揭示了语义表示的难处：人没有电气部件，人刮胡子时有剃刀，剃刀有电气部件。若把“拥有某物”与“某物是身体的组成部分”混为同一种关系，规则之间就可能产生错误关联。能够处理真实常识的系统需要大量事实、规则及一致的关系定义；这里的规则系统依赖预先给定的规则与陈述。

1.5 人工特征与学习得到的表示

1.5.1 经典机器学习中的特征映射

经典机器学习通常先由人选定特征，再学习特征与输出之间的映射。例如，朴素贝叶斯 (naïve Bayes) 采用“给定类别后，各特征条件独立”的假设；逻辑回归 (logistic regression) 学习各特征对输出的权重，训练过程可以使用梯度下降。这两个例子说明，特征如何表示输入，与模型如何利用这些特征，是可以分别考虑的环节。

不同任务需要不同的特征。图像可以经过视觉特征提取再进行检测；音频可以转换成音频特征，用于识别说话人；文本特征可以用于文本分类、机器翻译和信息检索。特征必须保留与任务输出有关的信息，同时让后续映射更容易表达。

1.5.2 坐标表示能够改变分类边界的复杂度

图 1 中，蓝色点聚集在中心，绿色点分布在外围。在直角坐标 (x, y) 下，仅用一条直线难以将两者分开；图中的多条线性不等式是在尝试用若干半平面描述中心区域。

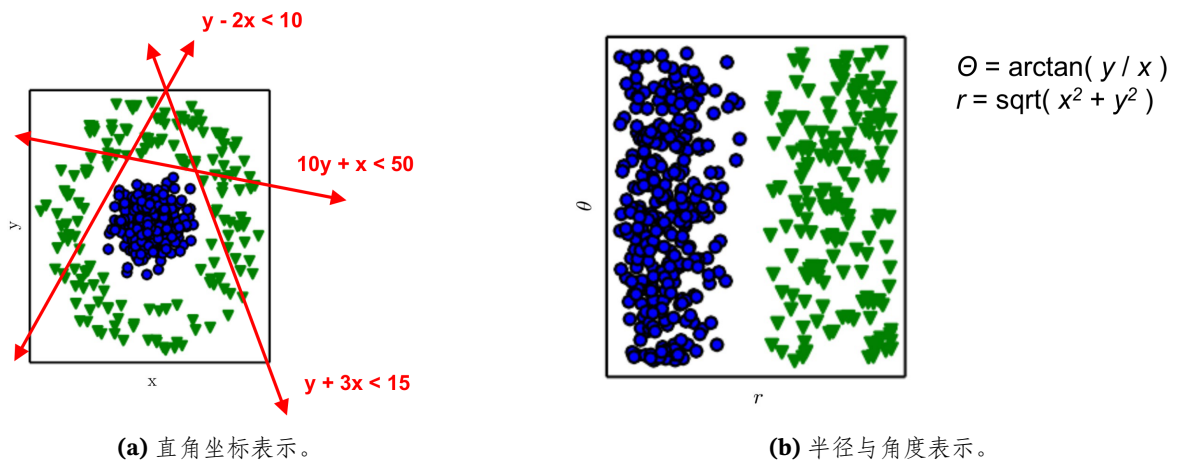


图 1 相同分类任务在不同特征坐标下具有不同的分离难度。

若改用半径 $r = \sqrt{x^2 + y^2}$ 和角度 θ ，中心点与外围点的差异就主要表现为 r 的大小。于是，用一个半径阈值就能描述图示的分离关系。这里有用的特征是“到中心的距离”，它把几何结构直接交给后续分类器。

图中的 $\theta = \arctan(y/x)$ 是简写；完整角度表示还需要区分象限，并处理 $x = 0$ 的情形。该例用于说明半径特征的作用，不依赖具体的角度实现。

1.5.3 从变化因素到层次化表示

人工指定“有车轮”作为识别汽车的特征，仍然要回答如何从像素中识别车轮。圆形轮廓、较暗的边缘等规则又会受到遮挡、观察角度、阴影和轮胎外观的影响。因此，选定一个看似合理的语义特征，并没有自动解决其底层检测问题。

变化因素（factors of variation）指影响观测数据的潜在因素。例如，年龄、性别、口音和说出的词语会影响语音；位置、颜色和太阳角度会影响汽车图像。一个观测通常同时受到多个因素影响，学习表示需要从这些混合变化中提取与任务相关的结构。

表示学习（representation learning）把特征本身也纳入学习过程。深度学习（deep learning）进一步用其他表示来表达新的表示，形成层次化特征。图像中的示意层次可以是输入像素、边缘、角点与轮廓、对象部件，最后是对象类别；较复杂概念由较简单概念组合而成。

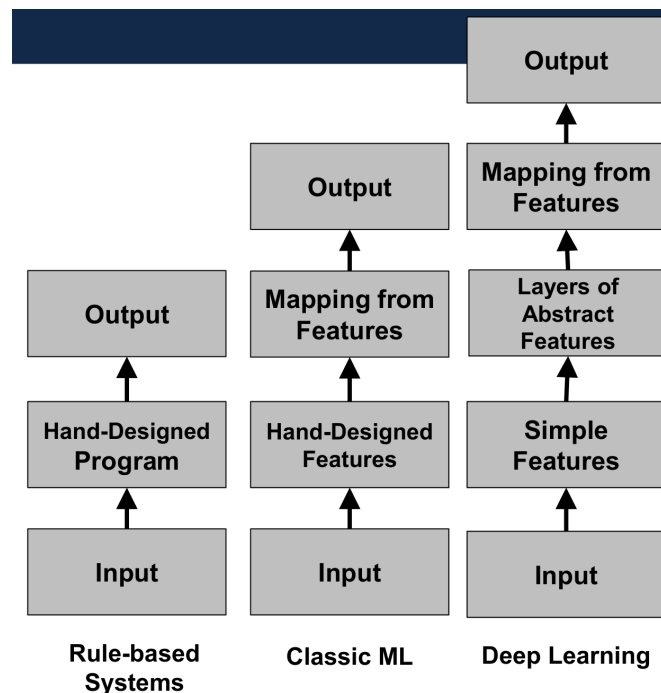


图2 规则系统、经典机器学习与深度学习中的输入到输出路径。深度学习在中间引入多层特征表示。

输入输出关系与中间表示

规则系统直接编码输入输出关系；经典机器学习学习人工特征到输出的映射；表示学习还学习特征的构造方式。深度学习通过层次组合表达特征。多层感知机中的隐藏层，为这种“由输入构造新表示，再根据新表示做判断”的过程提供了一个具体模型。

2 从感知机到多层网络

2.1 分类函数与单个感知机

分类 (classification) 把输入向量对应到某个离散类别。设输入 $x \in \mathbb{R}^N$, 类别数为 C , 未知的目标关系可以写成 $F: \mathbb{R}^N \rightarrow \{1, \dots, C\}$ 。模型使用可调参数 Θ , 构造近似这一关系的函数。对于后面的数字识别, 类别索引使用 $0, \dots, 9$; 这只是类别编号的变化。

一个感知机 (perceptron) 先计算输入的加权和, 再通过符号函数给出二分类结果。设权重 $w \in \mathbb{R}^N$ 、偏置 $b \in \mathbb{R}$, 则

$$u = w^T x + b = \sum_{j=0}^{N-1} w_j x_j + b, \quad \hat{y} = \text{sign}(u). \quad (3)$$

其中 $\Theta = \{w, b\}$; w_j 决定输入第 j 个分量对加权和的贡献, b 决定阈值的平移。这里的中间量 u 与最终的类别输出 $\hat{y}$ 分开记号, 便于区分点积和分类判断。

当 $u > 0$ 时取 $+1$, $u < 0$ 时取 -1 。决策边界是 $w^T x + b = 0$: 在二维输入空间中是一条直线, 在更高维中是一个超平面。本节的布尔例子使用 $0, 1$ 作为输入, 用 $-1, +1$ 编码假、真; 所选偏置保证样本不会恰好落在 $u = 0$ 上。

2.2 异或问题与隐藏表示

2.2.1 单条线性边界为什么无法表达异或

异或 (XOR) 在两个输入不同时为真, 相同时为假。因此, $(0, 1)$ 与 $(1, 0)$ 属于正类, $(0, 0)$ 与 $(1, 1)$ 属于负类。四个点的几何位置使同类点位于正方形的对角线上, 单条直线无法完成所需分离。

假设四个点均被严格正确分类, 则权重与偏置必须同时满足下列条件。

$$b < 0, \quad w_0 + w_1 + b < 0, \quad (4)$$

$$w_0 + b > 0, \quad w_1 + b > 0. \quad (5)$$

后两式相加得到 $w_0 + w_1 + 2b > 0$; 而前两式相加得到同一表达式小于 0 。两者矛盾。因此, 式 (3) 这种单个感知机不能完成该异或分类。

2.2.2 用 AND 与 OR 构造中间特征

AND 与 OR 分别可以由一个感知机实现。取如下权重与偏置：

$$h_{\text{OR}} = \text{sign}(x_0 + x_1 - 0.5), \quad (6)$$

$$h_{\text{AND}} = \text{sign}(x_0 + x_1 - 1.5). \quad (7)$$

两个感知机具有相同的输入权重 (1,1)，但使用不同偏置，因此具有不同判断阈值。原来的二维输入被转换成新坐标 $(h_{\text{AND}}, h_{\text{OR}})$ ：(0,0) 变为 $(-1, -1)$ ，两个“恰好一个输入为 1”的点都变为 $(-1, +1)$ ，(1,1) 变为 $(+1, +1)$ 。

这时，异或为真的样本已经集中到同一个表示位置。组合 $(+1, -1)$ 不可能出现，因为 AND 为真必然意味着 OR 为真。接下来的一个感知机只需要在这三个有效位置上完成分类。

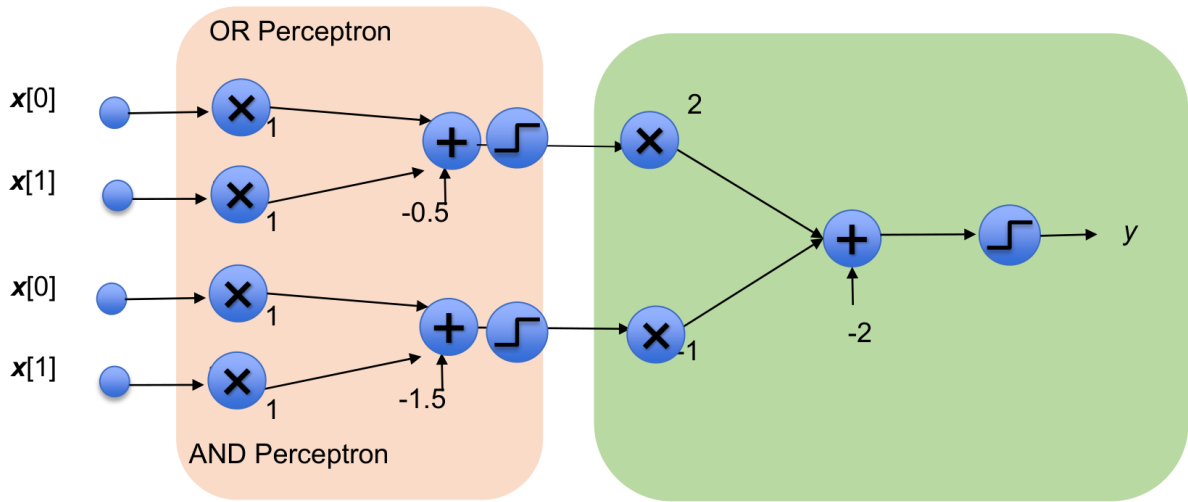


图 3 OR 与 AND 感知机构成隐藏层，再由带权组合和符号函数给出 XOR 输出。乘号旁的数值是连接权重，加法器旁的常数是偏置。

输出层取

$$\hat{y}_{\text{XOR}} = \text{sign}(2h_{\text{OR}} - h_{\text{AND}} - 2). \quad (8)$$

表 2 逐项核算这一表达式。对于只有一个输入为 1 的情形，输出层的加权和为 $2 \times 1 - (-1) - 2 = 1$ ，所以得到正类；其余两个情形分别得到 -3 和 -1 ，所以得到负类。

表 2 两层感知机的完整 XOR 计算。

x_0	x_1	h_{AND}	h_{OR}	$2h_{\text{OR}} - h_{\text{AND}} - 2$	输出
0	0	-1	-1	-3	-1
0	1	-1	+1	+1	+1
1	0	-1	+1	+1	+1
1	1	+1	+1	-1	-1

2.2.3 非线性变换如何改变下一层的问题

隐藏层的符号函数把输入变为新的特征位置。下一层的直线边界是在这个新表示空间中起作用，映射回原始输入后就能够实现单个感知机无法表达的分类关系。这里体现的是表示变化与分类边界之间的配合。

若两层之间没有激活函数，两次仿射变换可以合并。设 $h = W_1x + b_1$ ，输出加权和为 $W_2h + b_2$ ，则

$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2). \quad (9)$$

合并后的表达式仍只有一次仿射变换；若最终再接一个符号函数，仍然只产生一条线性决策边界。多层结构能够形成新表示，关键在于层间插入了合适的非线性函数。

XOR 例子的三个组成部分

第一层的权重与偏置定义 OR、AND 两个判断；隐藏层的符号函数生成新表示；输出层对这个表示加权、加偏置并再次判断。偏置与最终的符号函数共同构成式 (8) 的输出层。若仅用 $ah_{\text{OR}} + bh_{\text{AND}}$ ，不加偏置，则输入 00 与 11 对应的组合值互为相反数，无法同时得到 XOR 所需的负类输出。

2.3 全连接层、网络层次与参数维度

全连接层 (fully-connected layer) 让每个输出单元都连接到全部输入。若输入有 N 个分量、输出有 M 个分量，每个输出都需要一个长度为 N 的权重向量；把这些权重向量逐行排列，就得到权重矩阵 W 。

矩阵与索引约定

后续计算采用列向量： $x \in \mathbb{R}^N$ 、 $W \in \mathbb{R}^{M \times N}$ 、 $b \in \mathbb{R}^M$ 。 $W[i, j]$ 表示输入 j 到输出 i 的权重，即行对应输出，列对应输入。因此， $4 \rightarrow 3$ 层使用 3×4 权重矩阵；若按输入在行、输出在列排列权重，则须转置后参与这里的列向量乘法。连接数与参数量不因排列方向而改变。

在这一约定下，全连接层的加权和是 $fc = Wx + b$ ，其第 i 个分量为

$$fc[i] = \sum_{j=0}^{N-1} W[i, j]x[j] + b[i]. \quad (10)$$

每个输出再经过激活函数（activation function） ϕ ，得到 $h[i] = \phi(fc[i])$ 。对于逐元素激活， $h = \phi(Wx + b)$ 中的 ϕ 分别作用于向量各个分量。点积负责组合输入，偏置调节加权和的位置，激活函数给出非线性变换。

网络中的输入层（input layer）存放输入数据，隐藏层（hidden layer）产生中间表示，输出层（output layer）产生用于最终任务的输出。全连接网络包含多个这样的计算层时，就形成多层感知机（multilayer perceptron, MLP）。图中节点表示一个计算单元，连线表示参与加权求和的连接。

参数计数由全连接结构直接得到。输入为 N 维、输出为 M 维的一层有 MN 个权重与 M 个偏置，因此参数总数为 $M(N + 1)$ 。以 $4 \rightarrow 4 \rightarrow 3$ 的网络为例，在列向量约定下，第一层为 $W_1 \in \mathbb{R}^{4 \times 4}$ 、 $b_1 \in \mathbb{R}^4$ ，第二层为 $W_2 \in \mathbb{R}^{3 \times 4}$ 、 $b_2 \in \mathbb{R}^3$ 。第二层的 3 行分别生成 3 个输出，每一行都读取隐藏层的 4 个分量。整个网络共有 $4 \times 4 + 4 + 3 \times 4 + 3 = 35$ 个可调参数。

2.4 手写数字识别网络的具体结构

MNIST 手写数字识别例子以 28×28 的灰度图像为输入，输出数字 0 至 9 中的一个类别。给出的训练数据有 60,000 张带有人类标注的图像。把一张图像的像素按固定顺序展开，就得到长度为 $28 \times 28 = 784$ 的输入向量。

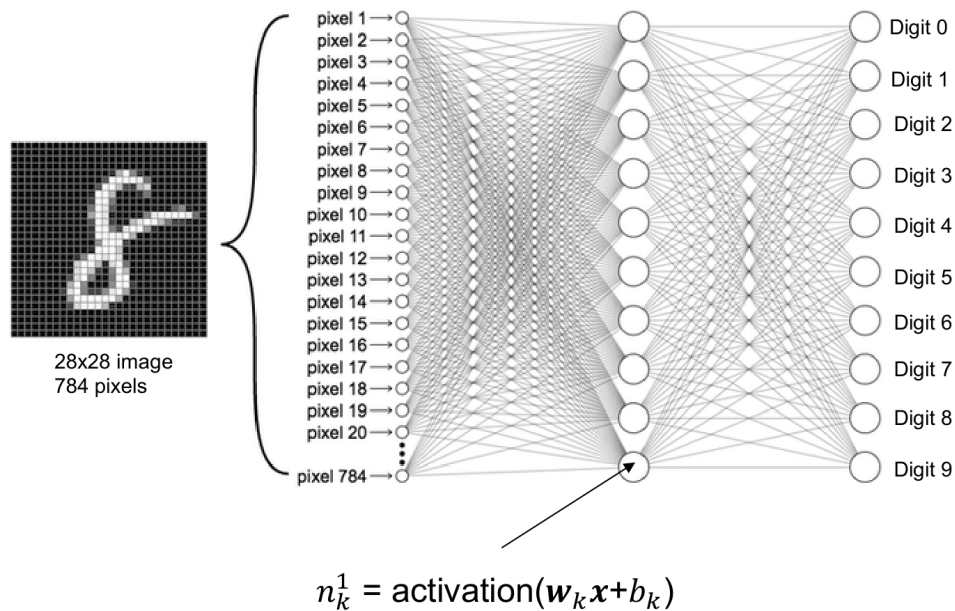


图 4 784 → 10 → 10 的手写数字识别网络。每个隐藏单元对 784 个像素加权求和，并经过激活函数；输出端有 10 个类别分量。

该例包含 784 个输入节点、10 个隐藏节点和 10 个输出节点。两层参数的形状与数量列在表 3 中。输入层不额外引入一组权重；参数属于输入到隐藏层、隐藏层到输出层这两次可调变换。

表 3 数字识别网络的参数维度与计数。

变换	权重形状	偏置形状	权重数	参数合计
784 → 10	10 × 784	10 × 1	7,840	7,850
10 → 10	10 × 10	10 × 1	100	110
总计			7,940	7,960

XOR 例子中，权重可以根据逻辑规则直接构造。数字识别中，需要用带标签的数据自动调整这 7,960 个参数，使输入像素与正确类别之间的关系编码到网络中。训练所需的两个进一步组成部分是：如何从连续输出获得类别信息，以及如何用输出误差计算参数的调整方向。

3 前向计算与分类输出

3.1 输入、分数、概率与类别的区别

前向传播 (forward propagation) 按照网络的连接方向依次求值。给定输入 x 和参数 Θ ，先计算各层加权和与激活值，再获得输出分量。分类任务还需要根据这些输出选择一个类别。

反向传播 (backpropagation) 则需要真实标签，用损失函数衡量输出与目标的差异，并计算损失对参数的导数。训练包含前向求值、损失计算、反向求导和参数更新；前向求值也是训练的一部分。推断时使用已经确定的参数产生输出。

表 4 两层数字识别网络的主要符号及其含义。

符号	含义	数字识别例子的形状
x	一张图像展开后的输入向量	784×1
fc_1, y	第一层加权和与激活输出	10×1
fc_2, z	第二层加权和与激活输出	10×1
k	归一化后的类别概率向量	10×1
$T, \hat{T}$	真实数字标签与最终预测标签	标量类别编号
Θ	两层的权重与偏置集合	共 7,960 个参数
E	一个带标签输入对应的损失	标量

两层数字识别网络在每层加权和之后均使用 **sigmoid**，再由 **softmax** 将第二层激活值转换为类别概率：

$$fc_1 = W_1 x + b_1, \quad y = \sigma(fc_1), \quad (11)$$

$$fc_2 = W_2 y + b_2, \quad z = \sigma(fc_2), \quad (12)$$

$$k = \text{softmax}(z), \quad \hat{T} = \arg \max_{0 \leq i < C} k[i]. \quad (13)$$

这里的 y 是隐藏层向量， z 是传入 **softmax** 的分数向量， k 才是满足归一化要求的类别概率； $\hat{T}$ 是离散类别编号。**arg max** 的作用是选出最大分量的索引，损失及其导数将在连续输出 k 上定义。

3.2 激活函数的形状与可求导性

3.2.1 符号函数与梯度传播

符号函数可以表达 XOR 构造中的硬阈值，但其输出在阈值两侧分别为常数。因此，在非零输入处导数为 0，在零点发生跳变。若直接使用普通链式法则传播梯度，大部分位置都会遇到这个零导数，难以提供连续调整权重所需的信号。

训练中的激活函数既决定前向变换，也决定反向传播时乘上的局部导数。激活值能否随输入的微小变化而改变，是理解这种梯度信号的重要出发点。

3.2.2 Sigmoid：输出值可以直接用于求导

Sigmoid，也称逻辑函数（logistic function），定义为

$$\sigma(u) = \frac{1}{1 + e^{-u}}, \quad \sigma : \mathbb{R} \rightarrow (0, 1). \quad (14)$$

当 u 增大时，输出从接近 0 平滑增加到接近 1，并在 $u = 0$ 时取 1/2。其导数可以写成已计算输出的简单函数：

$$\sigma'(u) = \frac{e^{-u}}{(1 + e^{-u})^2} \quad (15)$$

$$= \frac{1}{1 + e^{-u}} \left(1 - \frac{1}{1 + e^{-u}} \right) = \sigma(u)(1 - \sigma(u)). \quad (16)$$

因此，前向已经算得 $s = \sigma(u)$ 时，反向计算只需要 $s(1 - s)$ ，无需重新求一次指数函数。这也说明为什么保存前向激活值对训练有用。

由 $s \in (0, 1)$ 可直接得到 $0 < \sigma'(u) \leq 1/4$ ；在 s 接近 0 或 1 时导数接近 0。当激活值接近区间两端时，这一局部因子会减小传回的梯度。

3.2.3 ReLU 与 softplus

整流线性单元（rectified linear unit, ReLU）定义为

$$\text{ReLU}(u) = \max(0, u). \quad (17)$$

负输入被映射为 0，正输入保持不变。它的前向计算不需要指数函数，因此具有较简单的计算形式。由分段定义可得，在 $u < 0$ 时导数为 0，在 $u > 0$ 时导数为 1；零点没有普通导数。

实现反向传播时，需要单独约定零点处采用的梯度值。

Softplus 是 ReLU 的一种平滑近似：

$$\text{softplus}(u) = \ln(1 + e^u), \quad \frac{d \text{softplus}(u)}{du} = \frac{e^u}{1 + e^u} = \sigma(u). \quad (18)$$

这也解释了“softplus 是逻辑函数的一个原函数”的说法。Sigmoid、ReLU 与 softplus 的形状和计算代价不同；具体网络需要选择与其任务和训练方式相适应的函数。

一层中的两类计算

$Wx + b$ 通过矩阵运算组合输入，逐元素激活 $\phi(\cdot)$ 再改变每个输出分量。反向传播时，矩阵部分与激活部分分别贡献自己的导数，并按照链式法则相乘。激活函数的前向值与局部导数因此需要配套考虑。

3.3 Softmax 与互斥类别的概率

3.3.1 归一化的类别输出

对于 C 个互斥类别，模型输出的各个类别概率应当非负，且总和为 1。对每个分量独立使用 sigmoid，虽然能把各分量限制在 $(0, 1)$ ，却不会自动保证总和为 1；ReLU 也不提供这种归一化。因此，需要把分数向量进一步转换为类别概率向量。

给定 $z = (z[0], \dots, z[C-1])^T$ ，softmax 定义为

$$k[i] = \frac{e^{z[i]}}{\sum_{j=0}^{C-1} e^{z[j]}}, \quad i = 0, \dots, C-1. \quad (19)$$

每个分量的分子都为正，分母又是全部分子的总和，因此 $k[i] > 0$ 且 $\sum_i k[i] = 1$ 。较大的分数对应较大的概率；最终分类可以取 $\hat{T} = \arg \max_i k[i]$ 。

取 $z = (0.2, 0.4, 0.8)^T$ ，则

$$e^z \approx (1.2214, 1.4918, 2.2255)^T, \quad k \approx (0.2473, 0.3021, 0.4506)^T. \quad (20)$$

采用从 0 开始的类别编号时，预测类别为 2。分数 $z[2] = 0.8$ 和概率 $k[2] \approx 0.4506$ 是两个不同的量：概率需要把这一分数与全部其他类别的分数共同归一化。

归一化还带来平移不变性：对所有分数加同一个常数 c ，分子和分母都乘以 e^c ，因此 $\text{softmax}(z + c\mathbf{1}) = \text{softmax}(z)$ 。这表明输出取决于类别之间的相对分数。

有界分数对概率范围的限制

两层 sigmoid 网络的每个分数均满足 $0 < z[i] < 1$ 。当类别数 $C \geq 2$ 时，

$$k[i] = \frac{1}{1 + \sum_{j \neq i} e^{z[j]-z[i]}} < \frac{1}{1 + (C-1)e^{-1}} = \frac{e}{e + C - 1}.$$

因此， $C = 10$ 时任意类别的概率都小于约 0.232。这一上界来自 softmax 之前的 sigmoid 对分数范围的限制；它不限制通过最大概率索引得到的分类准确率。

3.3.2 Softmax 的导数为什么包含其他类别

改变一个分数 $z[m]$ 会同时改变它自己的分子，以及所有类别共享的分母。因此， $k[i]$ 不仅依赖 $z[i]$ ，还依赖其他类别的分数。这个耦合关系使 softmax 的导数成为一个矩阵。

令 $S = \sum_j e^{z[j]}$ 。用商法则展开，有

$$\frac{\partial k[i]}{\partial z[m]} = \frac{\delta_{i,m} e^{z[i]} S - e^{z[i]} e^{z[m]}}{S^2} \quad (21)$$

$$= k[i](\delta_{i,m} - k[m]), \quad (22)$$

其中 $\delta_{i,m}$ 是克罗内克符号 (Kronecker delta): $i = m$ 时为 1，否则为 0。这就给出两种情况：

$$\frac{\partial k[i]}{\partial z[m]} = \begin{cases} k[i](1 - k[i]), & i = m, \\ -k[i]k[m], & i \neq m. \end{cases} \quad (23)$$

自身分数上升会提高自身概率；某个其他类别的分数上升，则会降低当前类别的概率。这种联动来自总概率固定为 1 的要求。

令雅可比矩阵 $J_{\mathbf{k}}[i, m] = \partial k[i] / \partial z[m]$ ，则

$$J_{\mathbf{k}} = \text{diag}(\mathbf{k}) - \mathbf{k}\mathbf{k}^T. \quad (24)$$

若已经知道 $g_k[i] = \partial E / \partial k[i]$ ，链式法则给出

$$g_z[m] = \frac{\partial E}{\partial z[m]} = \sum_i g_k[i] \frac{\partial k[i]}{\partial z[m]} \quad (25)$$

$$= k[m] \left(g_k[m] - \sum_i k[i] g_k[i] \right), \quad (26)$$

$$g_z = k \odot (g_k - (k^T g_k) \mathbf{1}). \quad (27)$$

符号 $\odot$ 表示相同形状向量的逐元素乘法， $\mathbf{1}$ 表示全 1 向量。式 (27) 只需要一个内积与逐元素运算，就能完成这个导数传播；它与显式构造整个 $C \times C$ 雅可比矩阵后相乘的结果相同。

激活函数与 softmax 的作用范围

Sigmoid 与 ReLU 在这里逐元素作用于各个单元；softmax 同时读取一个样本的所有类别分数，并把它们耦合成归一化概率。前者的局部导数可以逐元素相乘，后者的导数还包含类别之间的交叉项。

4 损失函数、反向传播与参数更新

4.1 数字识别例子中的损失函数

损失函数 (loss function) 把模型输出与目标之间的差异转化为一个标量，使训练能够比较不同参数所产生的结果。对于真实数字标签 T 和输出概率 k ，一个候选损失是

$$E_1 = 1 - k[T]. \quad (28)$$

因为全部类别概率之和为 1， E_1 正好等于分配给所有错误类别的概率质量。它把全部错误类别同等对待，只关心正确类别分量的大小。

数字识别例子实际选用的另一种损失利用类别编号的数值差异：

$$E = \sum_{j=0}^{C-1} k[j] (j - T)^2. \quad (29)$$

定义 $q[j] = (j - T)^2$ ，可将其简写为 $E = k^T q$ 。正确类别的代价 $q[T] = 0$ ；距离目标数字更远的类别受到更大的惩罚。例如目标为 3 时，类别 2 的代价为 1，而类别 9 的代价为 36。这个数值距离是该例明确采用的损失设计。

式 (29) 可以解释为按 k 加权的数字平方距离。若随机变量 J 以概率 $k[j]$ 取值 j , 则 $E = \mathbb{E}[(J - T)^2]$ 。因此, 先求平方距离再求加权和, 与先求预测数字的均值再平方, 是不同的计算。例如 $k = (0.2, 0.5, 0.3)^T$ 、 $T = 1$ 时, 该损失为 $0.2 \times 1 + 0.5 \times 0 + 0.3 \times 1 = 0.5$; 预测数字的均值却是 1.1, 其与目标之差的平方只有 0.01。这里需要保留式 (29) 中的运算顺序。

4.2 从损失求到 softmax 输入的梯度

目标标签固定时, $q[j] = (j - T)^2$ 是常数。损失对概率输出的导数首先是

$$\frac{\partial E}{\partial k[i]} = q[i] = (i - T)^2. \quad (30)$$

再用式 (26), 得到

$$\frac{\partial E}{\partial z[m]} = \sum_i q[i] k[i] (\delta_{i,m} - k[m]) \quad (31)$$

$$= k[m] q[m] - k[m] \sum_i k[i] q[i] \quad (32)$$

$$= \boxed{k[m](q[m] - E)}. \quad (33)$$

向量形式是 $\mathbf{g}_z = k \odot (q - E\mathbf{1})$ 。对于目标类别 $m = T$, 有 $\mathbf{g}_z[T] = -k[T]E \leq 0$; 沿负梯度方向调整该分数, 会推动其上升。对于其他类别, 调整方向由该类别的代价 $q[m]$ 相对于当前平均代价 E 的大小决定。

继续使用 $\mathbf{z} = (0.2, 0.4, 0.8)^T$ 的三类别例子, 并取 $T = 1$, 则 $\mathbf{q} = (1, 0, 1)^T$, 有

$$E \approx 0.697936, \quad \mathbf{g}_z \approx (0.074703, -0.210821, 0.136118)^T. \quad (34)$$

梯度分量之和为 0, 可以由公式直接核验: $\sum_m k[m](q[m] - E) = E - E = 0$ 。这与 softmax 对分数共同平移不敏感的性质一致。

4.3 梯度下降与链式法则

梯度 (gradient) 记录损失对各个参数的局部变化率。设迭代编号为 r , 学习率为 $\epsilon > 0$, 一次梯度下降 (gradient descent) 更新写成

$$\Theta^{(r+1)} = \Theta^{(r)} - \epsilon \Delta\Theta, \quad \Delta\Theta = \nabla_{\Theta} E. \quad (35)$$

多个样本共同参与更新时, $\Delta\Theta$ 可以是相应样本梯度的累加量。学习率控制沿梯度方向移动的幅度。实际参数改变量是 $-\epsilon\Delta\Theta$, 而 $\Delta\Theta$ 表示更新时所用的梯度。

链式法则把复合函数的导数拆成相邻计算环节的导数。例如 Θ 先影响中间量 u , u 再影响损失 E , 就有 $\partial E / \partial \Theta = (\partial E / \partial u)(\partial u / \partial \Theta)$ 。在向量网络中, 同一个变量可能通过多个后继输出影响损失, 因此还需要把这些路径的贡献相加。

负梯度方向的局部下降性质来自一阶展开。令 $\mathbf{g} = \nabla E(\Theta)$, 在可微且步长足够小的情形下, $E(\Theta - \epsilon \mathbf{g}) \approx E(\Theta) - \epsilon \|\mathbf{g}\|^2$ 。这解释了更新式中的负号。

4.4 单层平方误差的完整更新

4.4.1 局部导数与参数导数

先考虑一个标量输入、标量输出的线性模型 $y = wx + b$, 目标值为 t , 损失取

$$E_{\text{sq}} = \frac{1}{2}(y - t)^2. \quad (36)$$

t 是该回归模型的数值目标。这个平方误差直接比较标量预测与目标, 与前面的概率加权数字距离损失具有不同含义。因子 $1/2$ 抵消求导时产生的 2 , 使输出梯度为 $\partial E_{\text{sq}} / \partial y = y - t$ 。同时, $\partial y / \partial w = x$, $\partial y / \partial b = 1$, 因此

$$\frac{\partial E_{\text{sq}}}{\partial w} = \frac{\partial E_{\text{sq}}}{\partial y} \frac{\partial y}{\partial w} = (wx + b - t)x, \quad (37)$$

$$\frac{\partial E_{\text{sq}}}{\partial b} = wx + b - t. \quad (38)$$

由此得到配套的权重与偏置更新:

$$w^+ = w - \epsilon(wx + b - t)x, \quad b^+ = b - \epsilon(wx + b - t). \quad (39)$$

权重梯度包含输入 x ，因为同样大小的权重变化，会在不同输入下造成不同大小的输出变化。偏置直接加到输出上，其局部导数是 1。

4.4.2 数值核算

取 $x = 2$ 、 $t = 3$ 、 $w = 0.5$ 、 $b = 0$ 、 $\epsilon = 0.1$ 。更新前， $y = 1$ ， $E_{sq} = 2$ ；输出梯度为 -2 ，权重梯度为 -4 ，偏置梯度为 -2 。同一次更新使用这些在旧参数处计算的梯度，得到 $w^+ = 0.9$ 、 $b^+ = 0.2$ 。再次前向计算，有 $y^+ = 2$ 、 $E_{sq}^+ = 0.5$ 。这个例子把前向求值、误差、局部导数、参数梯度与更新后的结果完整连接起来。

4.5 全连接层的三个梯度

4.5.1 每个权重如何影响损失

设一层输入 $x \in \mathbb{R}^N$ ，加权和 $fc = Wx + b \in \mathbb{R}^M$ ，并且已经从后续计算得到 $\delta[i] = \partial E / \partial fc[i]$ 。 δ 与该层输出加权和具有相同形状。由式 (10)，权重 $W[i, j]$ 对相应输出的局部导数是 $x[j]$ ，所以

$$\frac{\partial E}{\partial W[i, j]} = \frac{\partial E}{\partial fc[i]} \frac{\partial fc[i]}{\partial W[i, j]} = \delta[i]x[j]. \quad (40)$$

因此，一个权重的梯度需要两项信息：该权重连接的输入值，以及与输出单元对应的反向梯度。这正是前向信息与反向信息在参数计算中的交汇点。

偏置梯度与输入梯度同样由加权和的局部导数得到。偏置 $b[i]$ 对 $fc[i]$ 的导数为 1，因此 $\partial E / \partial b[i] = \delta[i]$ 。一个输入 $x[j]$ 会影响全部输出，所以它的梯度需要汇总所有输出路径：

$$\frac{\partial E}{\partial x[j]} = \sum_{i=0}^{M-1} \delta[i]W[i, j]. \quad (41)$$

三类梯度的矩阵形式为

$$\boxed{\nabla_W E = \delta x^T, \quad \nabla_b E = \delta, \quad \nabla_x E = W^T \delta.} \quad (42)$$

4.5.2 外积与维度核验

δ 的形状为 $M \times 1$ ， x^T 的形状为 $1 \times N$ ，因此外积 δx^T 的形状为 $M \times N$ ，正好与 W 一致。 $W^T \delta$ 的形状为 $N \times 1$ ，正好与输入一致。这种检查能够直接发现漏写转置或把两个不同梯度

混淆的问题。

对于多个输入分量、多个输出分量，式 (40) 没有把全部权重的导数压缩成一个标量。“输出梯度乘输入”在逐分量层面得到一个数，在整层层面则是产生完整权重梯度矩阵的外积。

4.6 两层数字识别网络的完整反向链

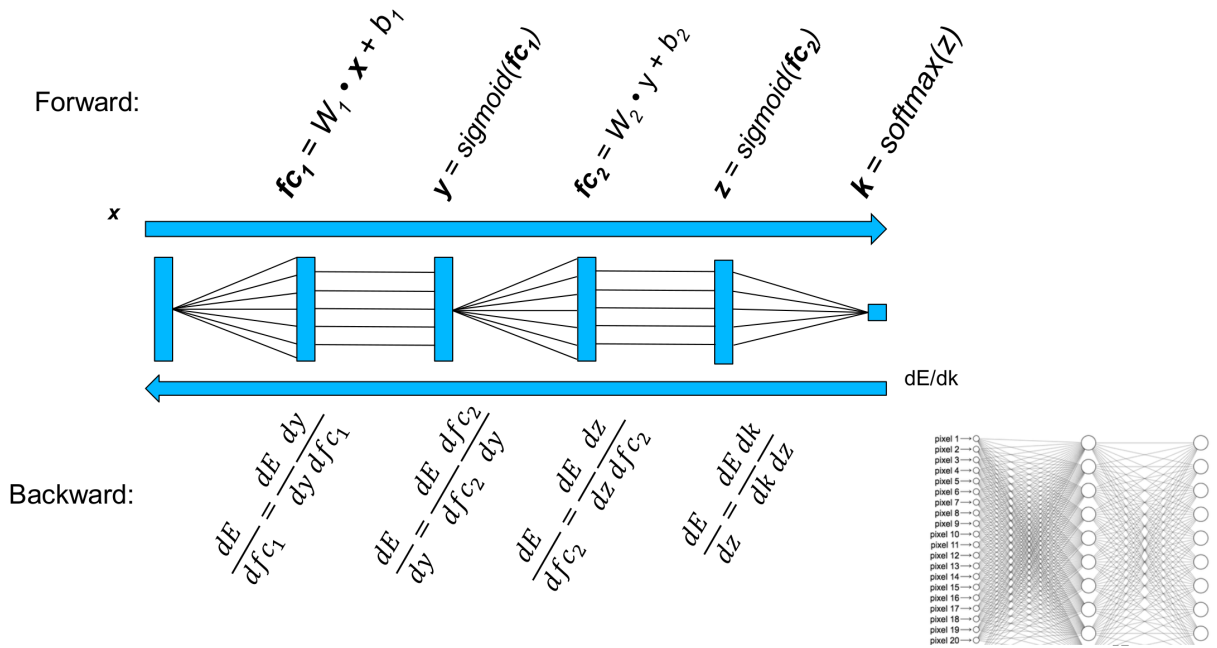


图 5 前向从输入逐层计算至概率输出，反向从损失对概率的导数逐层向前传播。图中的第二层也包含 sigmoid。

全部向量均为列向量， δ_ℓ 表示损失对第 ℓ 层加权之和的梯度。先在输出端计算 $q[i] = (i - T)^2$ 、 $E = k^T q$ 。概率层和第二次 sigmoid 的导数依次给出

$$g_z = k \odot (q - E\mathbf{1}), \quad (43)$$

$$\delta_2 = g_z \odot z \odot (\mathbf{1} - z). \quad (44)$$

δ_2 已经是对第二层加权之和 fc_2 的梯度，因此可以计算该层参数梯度：

$$\nabla_{W_2} E = \delta_2 y^T, \quad \nabla_{b_2} E = \delta_2. \quad (45)$$

同时，将梯度通过第二层传回隐藏输出，再通过隐藏层 **sigmoid**，得到

$$g_y = W_2^T \delta_2, \quad (46)$$

$$\delta_1 = g_y \odot y \odot (\mathbf{1} - y), \quad (47)$$

$$\nabla_{W_1} E = \delta_1 x^T, \quad \nabla_{b_1} E = \delta_1. \quad (48)$$

在 $784 \rightarrow 10 \rightarrow 10$ 网络中， $\nabla_{W_2} E$ 为 10×10 ， $\nabla_{W_1} E$ 为 10×784 ，两个偏置梯度都为 10×1 。所有梯度都与对应参数同形。

式 (44) 中的 $z \odot (\mathbf{1} - z)$ 来自第二次 **sigmoid**。只要采用式 (13) 所定义的网络结构，这个因子就属于完整的链式法则。损失直接作用于连续概率 k ，反向链不经过输出离散类别编号的 **arg max**。

4.7 前向缓存、梯度传递与参数更新的时序

计算 $\nabla_{W_1} E$ 需要第一层输入 x ，计算 $\nabla_{W_2} E$ 需要前向得到的隐藏输出 y 。对于 **sigmoid**，局部导数也可以直接由 y 和 z 得到；输出概率 k 则用于损失及其初始梯度。反向传播因此依赖前向过程中生成的信息。

同一次更新的梯度应当对应同一组参数。先用当前参数完成前向，再由这些中间结果完成反向，得到各层的参数梯度，最后统一进行 $W_\ell \leftarrow W_\ell - \epsilon \nabla_{W_\ell} E$ 与 $b_\ell \leftarrow b_\ell - \epsilon \nabla_{b_\ell} E$ 。传播到前一层时使用的 W_2^T ，仍是本次前向求值时的权重。

全连接层反向计算的分工

δx^T 计算当前层的权重梯度， δ 给出偏置梯度， $W^T \delta$ 则把梯度传给前一层。激活函数的导数在相应的激活节点处逐元素相乘。参数更新需要前向输入，跨层传递还需要当前层权重。

4.8 两层网络的数值核算

采用 $2 \rightarrow 2 \rightarrow 3$ 的小网络，保留“全连接、sigmoid、全连接、sigmoid、softmax”的结构，以及式 (29) 的损失。以下数值用于逐项检验计算公式。取输入、真实类别与初始参数为

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad T = 1, \quad W_1 = \begin{bmatrix} 0.1 & -0.2 \\ 0.3 & 0.25 \end{bmatrix}, \quad b_1 = \begin{bmatrix} 0.05 \\ -0.1 \end{bmatrix}, \quad (49)$$

$$W_2 = \begin{bmatrix} 0.4 & -0.3 \\ 0.1 & 0.2 \\ -0.2 & 0.5 \end{bmatrix}, \quad b_2 = \begin{bmatrix} 0 \\ 0.1 \\ -0.05 \end{bmatrix}. \quad (50)$$

网络共有 $2 \times 2 + 2 + 3 \times 2 + 3 = 15$ 个参数。第一层加权和为 $\text{fc}_1 = (-0.25, 0.7)^T$ ，继续逐层代入得到表 5。表中向量横向列出，均按列向量参与实际计算。

表 5 小网络一次前向计算的结果，保留六位小数。

中间量	分量
$y = \sigma(\text{fc}_1)$	(0.437823, 0.668188)
$\text{fc}_2 = W_2 y + b_2$	(-0.025327, 0.277420, 0.196529)
$z = \sigma(\text{fc}_2)$	(0.493669, 0.568914, 0.548975)
$k = \text{softmax}(z)$	(0.318978, 0.343906, 0.337116)
q 与 E	$q = (1, 0, 1)$, $E = 0.656094$

最大概率的类别已经是 1，但其他类别仍获得较多概率质量，因此损失仍明显大于 0。反向计算得到

$$g_z \approx (0.109698, -0.225635, 0.115936)^T, \quad (51)$$

$$\delta_2 \approx (0.027420, -0.055337, 0.028706)^T, \quad (52)$$

$$\delta_1 \approx (-0.00007552, -0.00109537)^T. \quad (53)$$

相应的两个权重梯度是

$$\nabla_{W_2} E \approx \begin{bmatrix} 0.012005 & 0.018322 \\ -0.024228 & -0.036976 \\ 0.012568 & 0.019181 \end{bmatrix}, \quad \nabla_{W_1} E \approx \begin{bmatrix} -0.00007552 & -0.00015104 \\ -0.00109537 & -0.00219074 \end{bmatrix}. \quad (54)$$

偏置梯度分别等于 δ_2 和 δ_1 。以 $\epsilon = 0.1$ 同时更新全部权重与偏置后，重新前向计算得到 $E^+ \approx 0.655334$ 。这个下降量来自全部 15 个参数的共同调整。

还可以对任意参数 θ_a 使用中心差分

$$\frac{\partial E}{\partial \theta_a} \approx \frac{E(\Theta + h e_a) - E(\Theta - h e_a)}{2h}, \quad (55)$$

其中 e_a 只在待检验参数位置为 1。用双精度、 $h = 10^{-6}$ 检查全部 15 个参数，解析梯度与中心差分的最大绝对差约为 8.3×10^{-11} 。该数值核验同时检查了两个 sigmoid 因子、softmax 的交叉项、权重外积与偏置梯度。

5 训练组织与并行计算

5.1 训练轮次与一次参数更新

一个训练轮次 (epoch) 表示遍历整个训练集一次。一次参数更新则表示计算一组梯度后，执行一次 $\Theta \leftarrow \Theta - \epsilon \Delta \Theta$ 。一个训练轮次中包含多少次更新，取决于每次更新使用多少个样本。

设训练集为 $\mathcal{D} = \{(x_s, T_s)\}_{s=1}^D$ 。采用梯度求和约定，全数据目标为

$$J_{\mathcal{D}}(\Theta) = \sum_{s=1}^D E_s(\Theta), \quad \nabla J_{\mathcal{D}}(\Theta) = \sum_{s=1}^D \nabla E_s(\Theta). \quad (56)$$

这里 D 是样本数， N 仍表示一个输入的维度。各个样本共享同一组网络参数，但具有各自的输入、标签、前向中间量及参数梯度。

5.2 全批量与小批量梯度下降

5.2.1 全批量：累积整个训练集后更新

全批量梯度下降 (full-batch gradient descent) 先将梯度累加器置零，再依次处理所有带标签样本。对每个样本，读取输入、前向计算输出、根据标签计算损失、反向求参数梯度，并把梯度加入累加器。整个训练集处理完毕后，执行一次参数更新。

整个累加过程中参数保持不变。因此，累加量正好对应式 (56) 在这一组参数处的梯度。一个 epoch 对应一次参数更新。这里的“完整梯度”是给定有限训练集目标的梯度。

5.2.2 小批量：按样本组交替求梯度和更新

小批量随机梯度下降（mini-batch stochastic gradient descent）把数据分成若干批量，每个批量单独清零梯度累加器、完成前向与反向、累加梯度并更新参数。设某批样本索引组成集合 $\mathcal{B}$ ，批量大小为 B ，对应梯度为

$$\Delta\Theta_{\mathcal{B}} = \sum_{s \in \mathcal{B}} \nabla E_s(\Theta), \quad \Theta^+ = \Theta - \epsilon \Delta\Theta_{\mathcal{B}}. \quad (57)$$

下一批样本使用更新后的参数继续计算。与全批量相比，这种方式在完整遍历训练集之前就会多次调整参数。批量大小还影响一次梯度估计利用的信息量，以及可同时处理的样本数量。

表 6 每次更新涉及的样本范围与更新次数。单样本情形作为小批量公式的 $B = 1$ 特例。

方式	一次更新使用的样本	一次完整遍历中的更新次数
全批量	全部 D 个	1
小批量	一组 B 个	D/B ，当 B 整除 D 时
单样本	1 个	D

以 60,000 个样本、 $B = 100$ 为例，一个 epoch 包含 600 个批量，因此有 600 次参数更新。若不整除且保留最后一个较小批量，更新次数为 $\lceil D/B \rceil$ 。

5.2.3 梯度求和与梯度求平均

上述更新采用梯度求和。若目标改为批量平均损失 $\bar{J}_{\mathcal{B}} = B^{-1} \sum_{s \in \mathcal{B}} E_s$ ，则梯度也需要除以 B 。这是同一批样本目标的尺度变化。

采用相同学习率时，平均梯度产生的参数改变量是求和梯度的 $1/B$ 。若要求两种写法产生相同的一步更新，应配套调整学习率。反向公式与参数更新之间，只需按所定义的目标处理一次这个归一化因子。

5.3 把小批量写成矩阵运算

按列组织一批样本后，逐样本的前向与反向过程可写成矩阵乘法、逐元素运算和归约。以下采用梯度求和约定。

5.3.1 按列组织样本的前向计算

把同一批中的 B 个输入作为矩阵 $X \in \mathbb{R}^{N \times B}$ 的各列。对于隐藏维度 M 、类别数 C 的两层网络，前向可以写成

$$F_1 = W_1 X + b_1 \mathbf{1}_B^T, \quad Y = \sigma(F_1), \quad (58)$$

$$F_2 = W_2 Y + b_2 \mathbf{1}_B^T, \quad Z = \sigma(F_2), \quad (59)$$

$$K_{:,s} = \text{softmax}(Z_{:,s}), \quad s = 1, \dots, B. \quad (60)$$

其中 $Y \in \mathbb{R}^{M \times B}$, $Z, K \in \mathbb{R}^{C \times B}$, $\mathbf{1}_B$ 是长度为 B 的全 1 向量。偏置项把同一个偏置向量加到每一列；softmax 对每个样本的类别列分别归一化。

5.3.2 多样本外积之和变成矩阵乘法

令 $G_2 \in \mathbb{R}^{C \times B}$ 的第 s 列是样本 s 的 δ_2 , $G_1 \in \mathbb{R}^{M \times B}$ 的第 s 列是其 δ_1 。将单样本外积按样本求和，可得

$$\nabla_{W_2} J_B = G_2 Y^T, \quad \nabla_{b_2} J_B = G_2 \mathbf{1}_B, \quad (61)$$

$$\nabla_{W_1} J_B = G_1 X^T, \quad \nabla_{b_1} J_B = G_1 \mathbf{1}_B. \quad (62)$$

例如， $G_1 X^T$ 的第 (i, j) 个元素是 $\sum_s \delta_{1,s}[i] x_s[j]$ ，正好是这一权重对批内全部样本的梯度累加量。传向隐藏层的梯度同样可以一次写成 $W_2^T G_2$ ，随后与 $Y \odot (\mathbf{1} - Y)$ 逐元素相乘。

表 7 批量网络中的主要计算形式。由前向与梯度矩阵公式直接归纳。

计算阶段	代表表达式	运算结构
全连接前向	$W_1 X$ 、 $W_2 Y$	矩阵乘法
激活与局部导数	$\sigma(F_1)$ 、 $Y \odot (\mathbf{1} - Y)$	逐元素计算
Softmax 与损失	每列指数和、加权代价和	按类别归约
跨层反向	$W_2^T G_2$	矩阵乘法
权重梯度	$G_2 Y^T$ 、 $G_1 X^T$	矩阵乘法
偏置梯度	$G_\ell \mathbf{1}_B$	按样本归约

批量把多个向量计算组织到同一个矩阵运算中，从而暴露样本之间的并行性。与此同时，激活值和反向梯度也多出一个大小为 B 的样本维度，需要保存更多中间数据。批量大小因此同时关联梯度估计、并行组织与中间存储；实际吞吐还取决于矩阵尺寸、存储访问与硬件资源。

6 泛化、模型容量与任务适配

6.1 训练集、验证集与测试集

泛化 (generalization) 是模型在新输入上仍能产生合适结果的能力。训练集用于计算参数更新；验证集用于选择设置和停止训练的位置；测试集用于检查模型对新输入的表现。三者在训练与评价过程中的用途不同。

训练损失持续下降，说明模型在参与参数更新的数据上拟合得更好。图 6 同时给出训练损失与验证损失随 epoch 变化的曲线：训练损失继续趋近于零，而验证损失先下降，随后趋于平缓并缓慢上升。这个变化说明，继续改进训练集上的拟合，可能无法同步改进新数据上的结果。

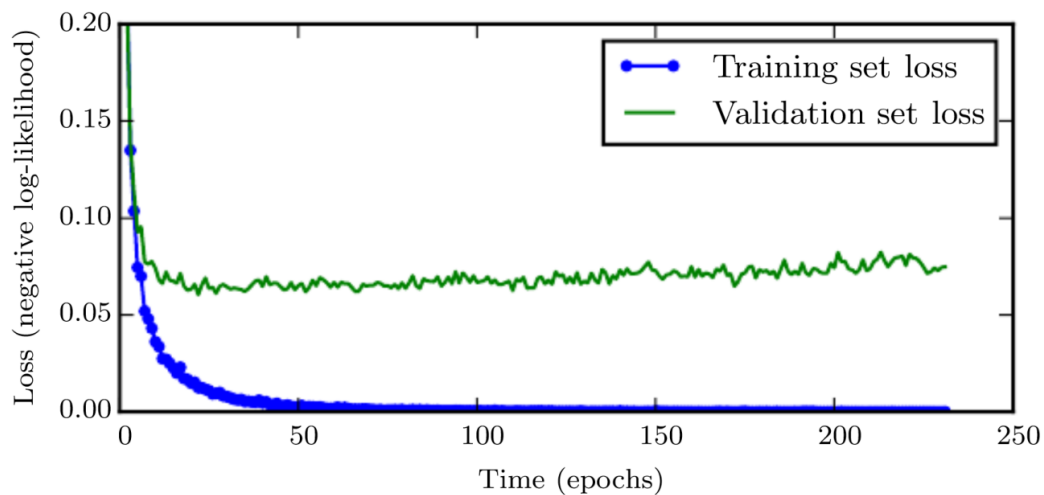


图 6 训练损失与验证损失随训练轮次的变化。纵轴为负对数似然损失；该曲线用于说明泛化现象，与前面数字距离损失的数值例题分开讨论。

验证数据为训练停止位置提供依据：参数更新继续使用训练集，停止点与设置则根据验证表现选择。测试数据用于最终检查泛化。由此，优化训练目标、选择模型和评估新输入表现形成相互关联而职责明确的环节。

6.2 模型容量、欠拟合与过拟合

6.2.1 多项式拟合的类比

模型容量 (capacity) 描述模型可以表达多复杂的关系。一个简单模型可能连训练样本中的主要变化都无法拟合, 表现为欠拟合 (underfitting)。增加表达能力后, 模型可以捕捉这些变化; 若过于贴合训练样本的具体细节, 则可能在未见输入上出现较大误差, 表现为过拟合 (overfitting)。

多项式拟合提供了一个直观例子。过低阶的曲线难以追随数据趋势; 合适的阶数能够表达主要关系; 过高阶的曲线可以更加贴近已知点, 同时在已知点之间或样本范围之外产生较大振荡。这个类比说明, 训练误差的改善需要结合新数据表现来判断。

6.2.2 训练轮次与模型容量是两个观察轴

图 7 的横轴是模型容量, 纵轴是误差。示意曲线中, 容量增加时训练误差下降, 而泛化误差在合适容量附近较低, 继续增大容量后又上升。两条曲线的差异称为泛化差距 (generalization gap)。

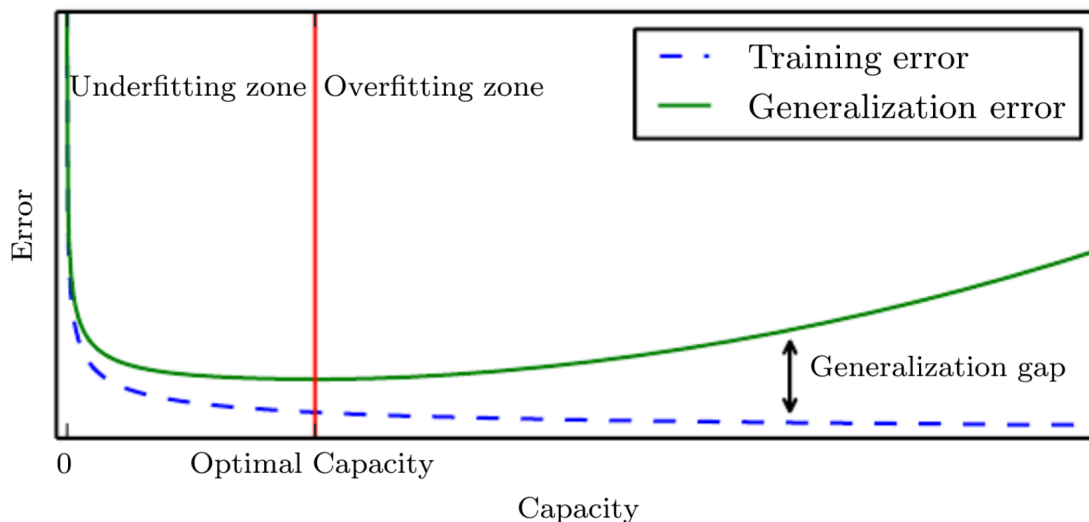


图 7 模型容量与训练误差、泛化误差之间的示意关系。横轴是容量, 不是训练轮次。

训练轮次衡量同一训练过程推进了多久, 模型容量描述结构能够表达多复杂的关系。前一幅曲线用于理解训练停止位置, 后一幅曲线用于理解容量选择。两者都需要关注训练数据之外的表现, 但对应的可调对象不同。

6.3 从权重图像理解学习到的特征

一层直接连接图像像素的网络，其某个单元具有 784 个输入权重。按照像素展开时的顺序，把这些权重重新排列成 28×28 的网格，就可以观察这个单元如何对不同像素位置加权。这种可视化把权重向量重新映射回输入的空间结构。

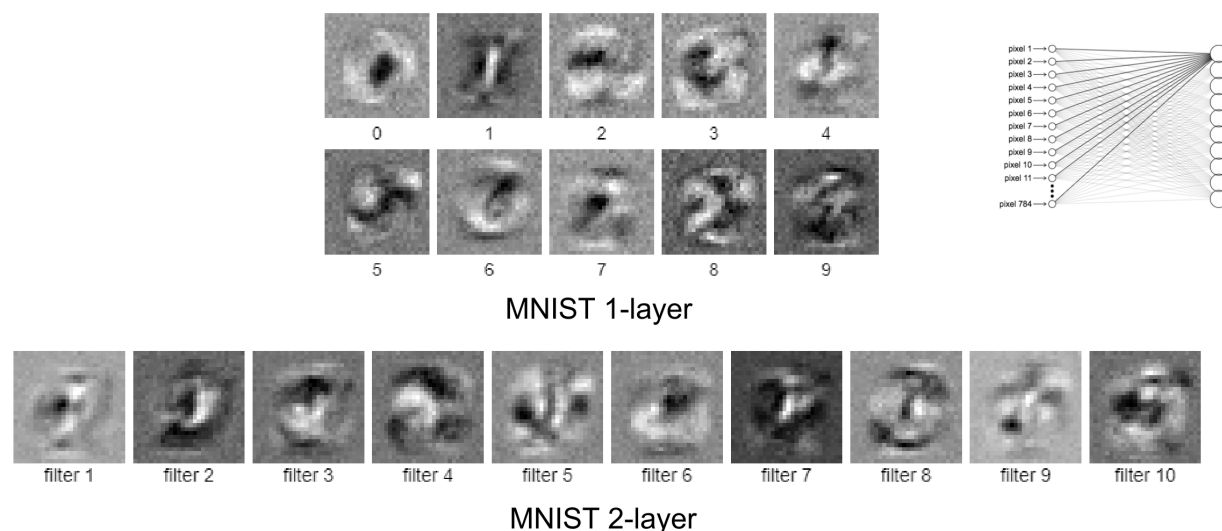


图 8 单层 MNIST 网络的类别权重图与两层网络的特征权重图。上方以数字 0-9 标注，下方以特征编号标注。

直接面向类别的权重图可以呈现与相应数字外观有关的结构。两层网络的隐藏单元则提供后续输出层要组合的中间特征，因此图中用特征编号而非数字类别为它们命名。这个区别与前面的表示学习一致：输出判断可以建立在多个中间特征的组合上。

6.4 无免费午餐定理的任务层面含义

无免费午餐定理 (no free lunch theorem) 讨论算法在广泛可能问题上的平均表现。一种概括性表述是：当对所有可能的输入生成分布取平均时，各种分类算法在未观测输入上的平均错误率相同。这里的关键限定是对所有可能分布取平均。

针对一个具体任务，模型面对的是具有特定结构的数据。网络结构、特征表示和训练设置都需要与这种结构相适配。因而，“从数据中学习”仍然包含任务相关的模型设计与选择；泛化评价用于检验这些选择是否在新输入上有效。平均性能结论依赖所选问题空间及平均方式，并不意味着各算法在某一个具体数据集上的表现相同。

7 关键公式与结构关系

7.1 从输入到参数更新

一个全连接层先计算 $\text{fc} = Wx + b$ ，再由激活函数产生下一层输入。多个这样的层组合后，网络获得逐层变换数据表示的能力。分类输出经 **softmax** 归一化为概率，损失函数再把概率与标签联系起来。

本讲两层网络的计算链

设 σ 为 **sigmoid**， T 为数字标签， $q[j] = (j - T)^2$ 。前向计算为

$$\begin{aligned} \text{fc}_1 &= W_1 x + b_1, & y &= \sigma(\text{fc}_1), \\ \text{fc}_2 &= W_2 y + b_2, & z &= \sigma(\text{fc}_2), \\ k &= \text{softmax}(z), & E &= k^T q. \end{aligned}$$

反向传播按相反顺序传递导数：

$$\begin{aligned} g_z &= k \odot (q - E\mathbf{1}), \\ \delta_2 &= g_z \odot z \odot (\mathbf{1} - z), \\ \delta_1 &= (W_2^T \delta_2) \odot y \odot (\mathbf{1} - y). \end{aligned}$$

因此 $\nabla_{W_2} E = \delta_2 y^T$ 、 $\nabla_{W_1} E = \delta_1 x^T$ ，偏置梯度分别为 δ_2 与 δ_1 。参数按 $\Theta \leftarrow \Theta - \epsilon \nabla_{\Theta} E$ 更新。

7.2 全连接层的三个梯度

对于 $W \in \mathbb{R}^{M \times N}$ 、 $x \in \mathbb{R}^N$ 和上游梯度 $\delta = \partial E / \partial \text{fc} \in \mathbb{R}^M$ ，三个基本关系为

$$\boxed{\nabla_W E = \delta x^T}, \quad \boxed{\nabla_b E = \delta}, \quad \boxed{\nabla_x E = W^T \delta}.$$

权重梯度把前向输入与反向导数组合成外积；输入梯度把误差信息传回上一层；偏置梯度直接接收该层导数。对一批样本求和后，外积累加成为矩阵乘法，偏置梯度成为沿样本维的求和。

表示学习建立输入与输出的关系；反向传播计算导数，梯度下降据此更新参数，小批量规定每次更新的样本范围。验证与测试用于评估模型对新输入的泛化能力。